

Spam Email Classifier

Project Overview

A machine learning application that automatically classifies emails as **Spam** or **Ham (Not Spam)** using natural language processing and supervised learning techniques. The system analyzes email content, extracts meaningful features, trains a classifier, and evaluates its performance using multiple metrics.

Features

- **Email Data Loading**: Load and preprocess email datasets (CSV format)
- **Text Preprocessing**: Tokenization, stop word removal, lowercasing, special character handling
- **Feature Extraction**: TF-IDF vectorization for converting text to numerical features
- **Multiple Classifiers**:
 - Naive Bayes (baseline)
 - Logistic Regression
 - Support Vector Machine (SVM)
- **Model Evaluation**: Accuracy, Precision, Recall, F1-Score, Confusion Matrix
- **Prediction Module**: Classify new emails in real-time
- **Performance Visualization**: Generate classification reports and metrics

Technologies/Tools Used

- **Python 3.8+**
- **scikit-learn**: Machine learning algorithms and evaluation metrics
- **pandas**: Data manipulation and analysis
- **numpy**: Numerical computing
- **nlTK**: Natural language processing (tokenization, stop words)

- ****pickle****: Model serialization

- ****pytest****: Unit testing

Project Structure

...

spam-email-classifier/

```
├── src/
│   ├── data_loader.py    # Dataset loading and exploration
│   ├── preprocessor.py   # Text preprocessing pipeline
│   ├── feature_extractor.py # TF-IDF feature extraction
│   ├── classifier.py     # Model training and prediction
│   ├── evaluator.py      # Performance evaluation
│   └── main.py           # Main execution pipeline
├── tests/
│   ├── test_preprocessor.py # Preprocessor unit tests
│   ├── test_feature_extractor.py # Feature extractor tests
│   └── test_classifier.py  # Classifier tests
├── data/
│   ├── emails.csv        # Training dataset
│   ├── training_data.pkl  # Preprocessed data cache
│   └── model.pkl         # Trained model cache
├── docs/
│   ├── README.md         # This file
│   └── statement.md      # Problem statement
└── .gitignore            # Git ignore patterns
```

```
└─ requirements.txt      # Python dependencies
...

```

Installation & Setup

Prerequisites

- Python 3.8 or higher
- pip (Python package manager)
- Git (for version control)

Steps to Install

1. ****Clone the repository****

```
` `` bash
git clone https://github.com/yourusername/spam-email-classifier.git
cd spam-email-classifier
...

```

2. ****Create a virtual environment****

```
` `` bash
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
...

```

3. ****Install dependencies****

```
` `` bash

```

```
pip install -r requirements.txt
```

```
```
```

#### 4. **\*\*Download NLTK data\*\*** (required for preprocessing)

```
```bash
```

```
python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords')"
```

```
```
```

### ## Running the Project

#### ### Basic Usage

```
```bash
```

```
# Run the complete pipeline (data loading → preprocessing → training → evaluation)
```

```
python src/main.py
```

```
# Run specific modules
```

```
python src/data_loader.py # Load and explore dataset
```

```
python src/preprocessor.py # Test preprocessing
```

```
python src/feature_extractor.py # Test feature extraction
```

```
python src/classifier.py # Train and test classifier
```

```
```
```

#### ### Interactive Classification

```
```python
```

```
from src.classifier import SpamClassifier
```

```
# Load trained model

clf = SpamClassifier()

clf.load_model('data/model.pkl')


# Classify new email

email_text = "You have won a free prize! Click here now."

prediction = clf.predict(email_text)

print(f"Classification: {prediction}") # Output: Spam or Ham
...
```

Testing Instructions

Run Unit Tests

```
```bash
```

```
Run all tests
```

```
pytest tests/ -v
```

```
Run specific test file
```

```
pytest tests/test_preprocessor.py -v
```

```
Run with coverage report
```

```
pytest tests/ --cov=src --cov-report=html
```

```
...
```

### ### Manual Testing

```
1. Start the application: `python src/main.py`
```

2. Review classification results and metrics
3. Analyze confusion matrix and classification report
4. Test with custom emails using the interactive module

## ## Data Format

### ### Input Dataset (emails.csv)

` `` ` csv

id,email\_subject,email\_body,label

1,"Great Offer","Buy now and save 50%!",spam

2,"Meeting Tomorrow","Hi, see you at 2pm",ham

3,"URGENT: Claim Your Prize","Click here to claim \$1000",spam

` `` `

### \*\*Columns:\*\*

- `id` : Unique email identifier
- `email\_subject` : Subject line of the email
- `email\_body` : Body/content of the email
- `label` : Target variable ("spam" or "ham")

### ### Train/Test Split

- Training set: 80% of data
- Test set: 20% of data
- Stratified split to maintain class balance

## ## Model Performance

### ### Expected Metrics (on test set)

- **Accuracy**: ~95-98%
- **Precision (Spam)**: ~94-97%
- **Recall (Spam)**: ~93-96%
- **F1-Score (Spam)**: ~93-96%

\*Note: Actual metrics depend on dataset quality and preprocessing parameters\*

## ## Screenshots / Results

### ### Sample Output

```

===== SPAM EMAIL CLASSIFIER =====

Dataset loaded: 5572 emails

Training set: 4457 emails (80%)

Test set: 1115 emails (20%)

===== MODEL EVALUATION =====

Classifier: Multinomial Naive Bayes

Accuracy: 0.9654

Precision (Spam): 0.9521

Recall (Spam): 0.9406

F1-Score (Spam): 0.9463

===== CONFUSION MATRIX =====

```

        Predicted
        Ham Spam
Actual Ham [948 12]
        Spam [17 138]
    \ \ \

```

```
## Configuration

```

```
### Hyperparameters (in `src/classifier.py` )

```

```
```python

```

```
TF-IDF Configuration

```

```
max_features = 5000

```

```
min_df = 2

```

```
max_df = 0.95

```

```
Classifier Configuration

```

```
random_state = 42

```

```
test_size = 0.2

```

```
```

```

Modify these values to tune model performance.

```
## Future Enhancements

```

- Deploy as REST API using Flask/FastAPI
- Add deep learning models (LSTM, CNN for text classification)
- Implement user feedback loop for continuous learning

- Add visualization dashboard
- Support for multiple languages
- Spam type classification (phishing, advertising, etc.)